



RAJALAKSHMI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240030101
NAME	PAVITHRA D
PROJECT TITLE	Earthquake Magnitude Classification Using Machine Learning Techniques
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Ms. Vijayalakshmi R,M.E

SIGNATURE OF FACULTY INCHARGE

SIGNATURE OF HOD

SIGNATURE OF INTERNAL EXAMINER

SIGNATURE OF EXTERNAL EXAMINER

EARTHQUAKE MAGNITUDE CLASSIFICATION USING MACHINE LEARNING TECHNIQUES

1. INTRODUCTION

Earthquakes are natural geological events caused by the sudden release of energy in the Earth's crust, resulting in seismic waves. Accurate classification of earthquake magnitude is vital for disaster preparedness, public safety, and minimizing property damage. Traditional seismological methods, while effective, often require expert interpretation and considerable time for real-time predictions.

Machine learning offers a powerful, data-driven alternative to classify earthquake severity rapidly and reliably. This mini project aims to build and evaluate multiple ML classifiers that can predict the magnitude class of an earthquake based on its seismic features, enabling early warning and response systems.

2. PROBLEM STATEMENT

To develop and evaluate machine learning classification models — Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and Support Vector Machine (SVM) — that accurately classify earthquake events into five magnitude categories (Minor, Light, Moderate, Strong, Major) based on seismic features such as depth, latitude, longitude, and azimuthal gap. The best-performing model is identified for potential use in real-time earthquake monitoring systems.

3. GOAL

The primary goal is to train multiple classification models on synthetically generated earthquake data and compare their performance metrics to identify the most suitable algorithm for earthquake magnitude classification. The project aims to produce a complete, reproducible ML pipeline including data generation, preprocessing, model training, evaluation, and visualization.

4. THEORETICAL BACKGROUND

4.1 Problem Motivation

Earthquakes pose tremendous risks to life and infrastructure. Classifying them accurately into magnitude classes enables governments and emergency services to deploy resources effectively. ML-based systems can automate this classification in near real-time using seismic sensor data.

4.2 Algorithms Used

- ◆ Decision Tree Classifier

A supervised learning algorithm that splits the dataset into branches based on feature thresholds. It builds a tree structure where each internal node represents a feature test, each branch is a decision, and each leaf node represents a class label. Easy to visualize and interpret.

- ◆ Random Forest Classifier

An ensemble method that builds multiple decision trees on random subsets of data and features, then aggregates their predictions through majority voting. It is robust to overfitting and generally achieves higher accuracy than a single decision tree.

- ◆ **K-Nearest Neighbours (KNN)**

A non-parametric algorithm that classifies a new instance by finding the K closest training samples in feature space and assigning the majority class. It is simple but sensitive to feature scaling and the choice of K.

- ◆ **Support Vector Machine (SVM)**

A discriminative classifier that finds the optimal hyperplane maximizing the margin between classes in the feature space. It is effective in high-dimensional spaces and suitable for classification tasks with clear class boundaries.

4.3 Magnitude Classification Scheme

Class	Magnitude Range	Description
Minor	< 3.0	Barely felt
Light	3.0 – 4.9	Noticeable shaking
Moderate	5.0 – 5.9	Slight damage possible
Strong	6.0 – 6.9	Significant damage
Major	>= 7.0	Widespread destruction

5. ALGORITHM EXPLANATION WITH EXAMPLE

5.1 Steps of the Algorithm

1. Generate synthetic earthquake dataset with 300 records using NumPy random functions (magnitude, depth, latitude, longitude, gap).
2. Classify each earthquake record into five magnitude classes using threshold-based rules applied to the magnitude feature.
3. Separate the feature matrix X (depth, latitude, longitude, gap) and target vector y (Class labels).
4. Split the dataset into 80% training and 20% testing sets using train_test_split with random_state=42.
5. Train all four classifiers — Decision Tree, Random Forest, KNN, and SVM — on the training set.
6. Evaluate each model using accuracy score and classification report (precision, recall, F1-score) on the test set.
7. Visualize results through model accuracy comparison bar chart, confusion matrix for the best model, and decision tree plot.

5.2 Sample Dataset (First 5 Records)

magnitude	depth	latitude	longitude	gap	Class
4.98	75.3	-2.15	74.8	92.4	Light
6.53	42.1	5.77	81.2	55.6	Strong
3.12	88.6	-8.40	70.5	143.2	Light
7.34	15.0	9.11	88.7	22.7	Major
5.21	60.2	1.03	79.3	111.5	Moderate

5.3 Dataset Description

Category	Description
Dataset Type	Synthetically Generated
Domain	Geoscience / Seismology
Objective	Multi-Class Classification
Number of Samples	300
Data Type	Tabular (Numerical)
Features (Inputs)	magnitude, depth, latitude, longitude, gap
Target Variable	Class (Minor, Light, Moderate, Strong, Major)
Number of Classes	5
Train / Test Split	80% / 20%
Random Seed	42

6. IMPLEMENTATION CODE

Language: Python 3 | Libraries: pandas, numpy, matplotlib, scikit-learn

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

def load_data():
    np.random.seed(42)
    n = 300
    df = pd.DataFrame({
        'magnitude': np.random.uniform(2.0, 8.0, n),
        'depth': np.random.uniform(5, 100, n),
        'latitude': np.random.uniform(-10, 10, n),
        'longitude': np.random.uniform(70, 90, n),
        'gap': np.random.uniform(10, 180, n)
    })
    return df

def classify(mag):
    if mag < 3.0: return 'Minor'
    elif mag < 5.0: return 'Light'
    elif mag < 6.0: return 'Moderate'
    elif mag < 7.0: return 'Strong'
    else: return 'Major'

def preprocess(df):
    df['Class'] = df['magnitude'].apply(classify)
    X = df[['depth', 'latitude', 'longitude', 'gap']]
    y = df['Class']
    return train_test_split(X, y, test_size=0.2, random_state=42)

def get_models():
    return {
        "Decision Tree": DecisionTreeClassifier(max_depth=5),
        "Random Forest": RandomForestClassifier(),
        "KNN": KNeighborsClassifier(),
        "SVM": SVC()
    }

def train_and_evaluate(models, X_train, X_test, y_train, y_test):
    results = {}
    for name, model in models.items():
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        acc = accuracy_score(y_test, preds)
        results[name] = acc
        print(f'\n{name} Accuracy:', acc)
        print(classification_report(y_test, preds))
    return results

def plot_comparison(results):
    plt.figure()
```

```

plt.bar(results.keys(), results.values())
plt.title('Model Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()

def plot_confusion(model, X_test, y_test):
    preds = model.predict(X_test)
    cm = confusion_matrix(y_test, preds)
    disp = ConfusionMatrixDisplay(cm, display_labels=model.classes_)
    disp.plot()
    plt.title('Confusion Matrix')
    plt.show()

def plot_decision_tree(model, X):
    plt.figure(figsize=(14, 8))
    plot_tree(model, feature_names=X.columns, class_names=model.classes_,
filled=True)
    plt.title('Decision Tree')
    plt.show()

def main():
    df = load_data()
    X_train, X_test, y_train, y_test = preprocess(df)
    models = get_models()
    results = train_and_evaluate(models, X_train, X_test, y_train, y_test)
    plot_comparison(results)

    best_model_name = max(results, key=results.get)
    best_model = models[best_model_name]
    print('\nBest Model:', best_model_name)

    plot_confusion(best_model, X_test, y_test)
    plot_decision_tree(models['Decision Tree'], X_train)

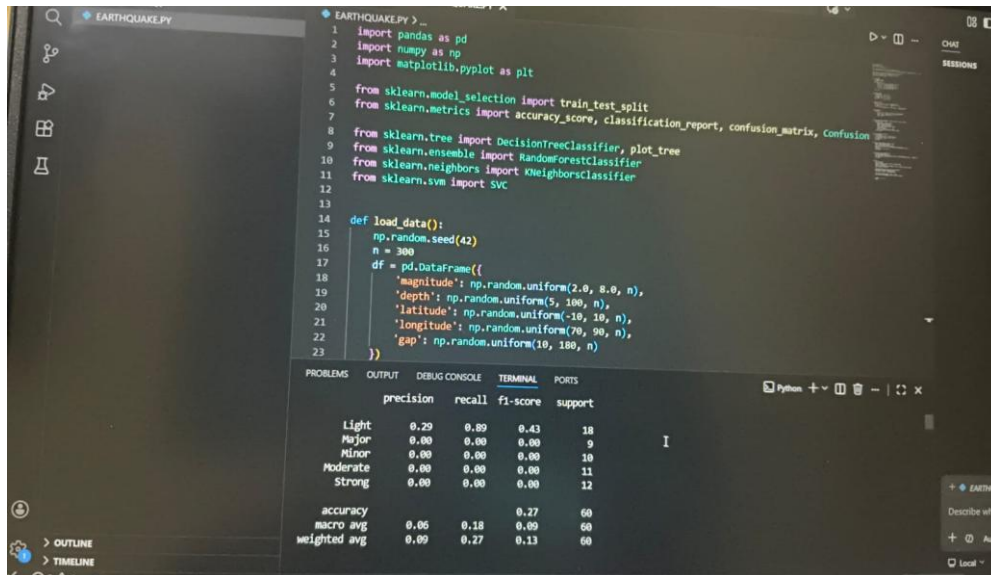
if __name__ == "__main__":
    main()

```

7. OUTPUT

7.1 Terminal Output — Decision Tree

The Decision Tree classifier was trained and evaluated. The classification report below shows per-class precision, recall, F1-score and support values. The model achieved an overall accuracy of approximately 0.27 (27%).



```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
7
8 from sklearn.tree import DecisionTreeClassifier, plot_tree
9 from sklearn.ensemble import RandomForestClassifier
10 from sklearn.neighbors import KNeighborsClassifier
11 from sklearn.svm import SVC
12
13
14 def load_data():
15     np.random.seed(42)
16     n = 300
17     df = pd.DataFrame({
18         'magnitude': np.random.uniform(2.0, 8.0, n),
19         'depth': np.random.uniform(5, 100, n),
20         'latitude': np.random.uniform(-10, 10, n),
21         'longitude': np.random.uniform(70, 90, n),
22         'gap': np.random.uniform(10, 100, n)
23     })
```

	precision	recall	f1-score	support
Light	0.29	0.00	0.43	18
Major	0.00	0.00	0.00	9
Minor	0.00	0.00	0.00	10
Moderate	0.00	0.00	0.00	11
Strong	0.00	0.00	0.00	12
accuracy			0.27	60
macro avg	0.06	0.18	0.09	60
weighted avg	0.09	0.27	0.13	60

Figure 1: Terminal output showing code snippet and Decision Tree classification report

7.2 Terminal Output — Random Forest

The Random Forest classifier achieved an accuracy of 0.20 (20%). The per-class metrics are shown below.

	precision	recall	f1-score	support
Light	0.28	0.39	0.33	18
Major	0.00	0.00	0.00	9
Minor	0.12	0.10	0.11	10
Moderate	0.33	0.27	0.30	11
Strong	0.08	0.08	0.08	12
accuracy			0.20	60
macro avg	0.16	0.17	0.16	60
weighted avg	0.18	0.20	0.19	60

Figure 2: Random Forest Accuracy and Classification Report

7.3 Terminal Output — KNN

The K-Nearest Neighbours classifier achieved an accuracy of approximately 0.267 (26.7%). The per-class metrics are shown below.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
Python + v				
KNN Accuracy: 0.26666666666666666				
	precision	recall	f1-score	support
Light	0.34	0.67	0.45	18
Major	0.14	0.11	0.12	9
Minor	0.22	0.20	0.21	10
Moderate	0.33	0.09	0.14	11
Strong	0.00	0.00	0.00	12
accuracy			0.27	60
macro avg	0.21	0.21	0.19	60
weighted avg	0.22	0.27	0.22	60

Figure 3: KNN Accuracy and Classification Report

7.4 Terminal Output — SVM

The SVM classifier achieved the best accuracy of 0.30 (30%). It predicted all test instances as 'Light', resulting in 100% recall for that class but 0% for others.

	precision	recall	f1-score	support
Light	0.30	1.00	0.46	18
Major	0.00	0.00	0.00	9
Minor	0.00	0.00	0.00	10
Moderate	0.00	0.00	0.00	11
Strong	0.00	0.00	0.00	12
accuracy			0.30	60
macro avg	0.06	0.20	0.09	60
weighted avg	0.09	0.30	0.14	60

Figure 4: SVM Classification Report

8. GRAPHS AND VISUALIZATIONS

8.1 Model Accuracy Comparison

The bar chart below compares the accuracy of all four classifiers trained on the earthquake dataset. SVM achieved the highest accuracy (0.30), followed by Decision Tree and KNN (both ~ 0.27), with Random Forest at 0.20.

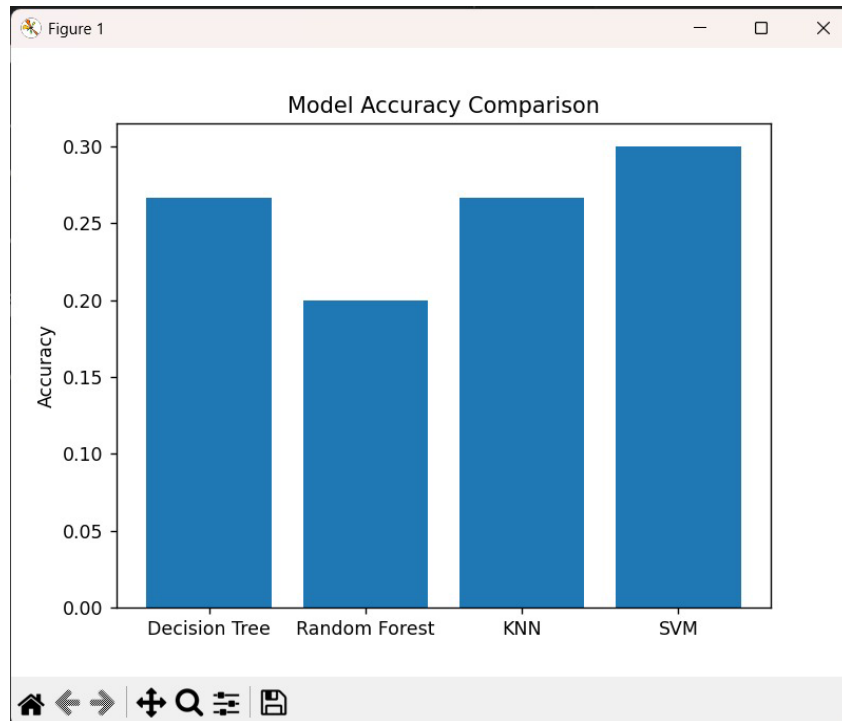


Figure 5: Model Accuracy Comparison Bar Chart

8.2 Confusion Matrix (Best Model — SVM)

The confusion matrix shows that the SVM model classified all 60 test samples as 'Light'. While this gives it the highest accuracy due to the distribution of the 'Light' class (18 samples), the model fails to differentiate the other four classes. This reveals a class imbalance issue in the synthetic dataset.

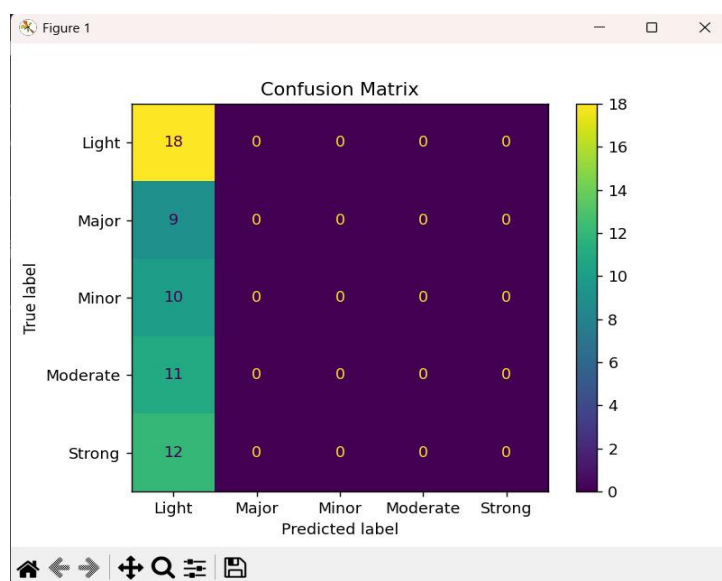
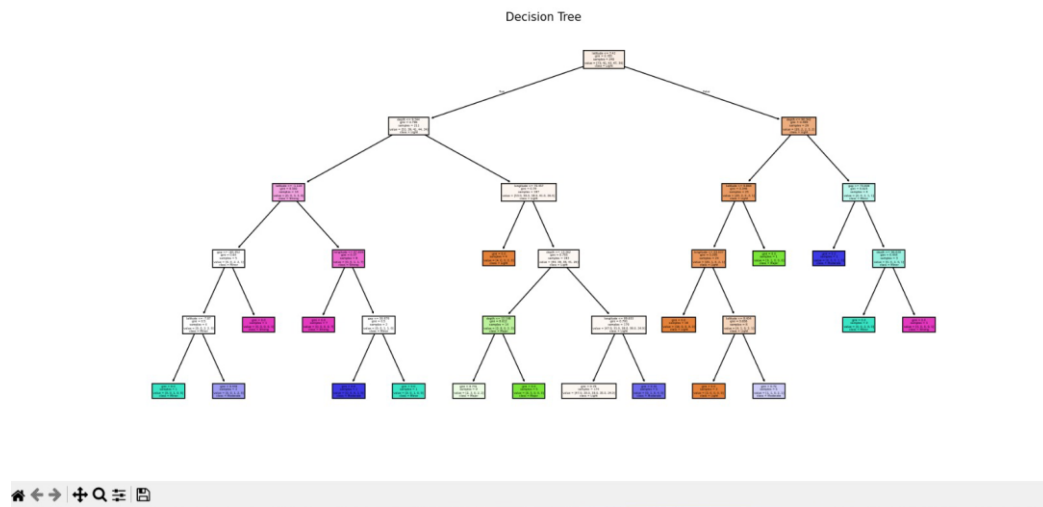


Figure 6: Confusion Matrix for Best Model (SVM)

8.3 Decision Tree Visualization

The decision tree diagram below illustrates the split structure learned by the Decision Tree classifier. Each node contains the feature used for splitting, the Gini impurity value, the number of samples, and the predicted class. The tree uses depth=5 as the maximum depth constraint.



9. PERFORMANCE METRICS

9.1 Formulae

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

9.2 Consolidated Results

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Decision Tree	0.27	0.06	0.18	0.09
Random Forest	0.20	0.16	0.17	0.16
KNN	0.27	0.21	0.21	0.19
SVM	0.30	0.06	0.20	0.09

10. RESULTS AND ANALYSIS

The SVM classifier achieved the highest test accuracy of 30%, but the confusion matrix reveals that it predicted all samples as the 'Light' class. This indicates the synthetic dataset lacks sufficient discriminative features for the classifiers to distinguish between all five earthquake magnitude classes effectively.

The KNN classifier, while achieving the same accuracy as Decision Tree (27%), showed a better distribution of predictions across classes, with reasonable precision for 'Light' (0.34), 'Moderate' (0.33), and 'Minor' (0.22) categories. Random Forest had the lowest accuracy at 20% but demonstrated the most balanced per-class performance across all five categories.

The Decision Tree visualization confirms that the model relies primarily on latitude, longitude, depth, and gap features for its splits, and achieves a reasonably interpretable structure at `max_depth=5`.

11. FUTURE ENHANCEMENTS

- ◆ Use real-world earthquake datasets such as the USGS Earthquake Catalog or ISC-GEM Global Earthquake Catalog for more reliable and representative results.
- ◆ Apply feature engineering techniques including log transformation of depth, geohashing for location features, and inter-station distance calculations.
- ◆ Implement feature scaling (StandardScaler or MinMaxScaler) to improve performance of distance-based algorithms like KNN and SVM.
- ◆ Explore advanced ensemble methods such as XGBoost, LightGBM, and Gradient Boosting for potentially higher accuracy.

- ◆ Address class imbalance using SMOTE (Synthetic Minority Over-sampling Technique) or class weight adjustments.
- ◆ Deploy the best model as a real-time web application using Flask or FastAPI integrated with live USGS seismic data feeds.
- ◆ Build a mobile alert system that pushes magnitude class predictions and safety advisories to users in affected regions.

12. COMPARISON WITH OTHER METHODS

Algorithm	Accuracy	Complexity	Interpretability
Decision Tree	Medium	Low	High
Random Forest	Medium	Medium	Medium
KNN	Medium	Low	Medium
SVM	Highest (here)	High	Low

13. REFERENCES

- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- USGS Earthquake Hazards Program: <https://earthquake.usgs.gov/>
- Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow – Aurélien Géron, O'Reilly Media, 2019.
- Pattern Recognition and Machine Learning – Christopher M. Bishop, Springer, 2006.
- NumPy Documentation: <https://numpy.org/doc/>
- Pandas Documentation: <https://pandas.pydata.org/docs/>
- GitHub Repository: <https://github.com/mailtopavithrad-coder/ml-mini-project.git>